Using oTree: A Practical Guide

Nidhi Gyan Pandey

2026-03-31

Table of contents

Welcome **4**
Prerequisites . . . 4
A Note on This Book . . . 4

Let's Get Started! **5**

Setting Up Visual Studio Code **7**
macOS . . . 7
Installation . . . 7
Windows . . . 7
Installation . . . 7
Installing Extensions . . . 8
How to Install an Extension . . . 8
Required Extensions . . . 8
Recommended Extensions . . . 8

Installing Python and oTree **10**
Step 1: Check if Python is Already Installed . . . 10
Step 2: Install Python (Only if Needed) . . . 11
macOS . . . 11
Windows . . . 11
Step 3: Install oTree . . . 11
Step 4: Verify oTree Installation . . . 12
Troubleshooting . . . 12

Session 1 **13**

Getting Started: VS Code, Projects, and Apps **15**
Opening VS Code and navigating to the right folder . . . 15
Opening the terminal in VS Code . . . 15
Navigating folders: `cd` . . . 16
Finding your working directory: `pwd` . . . 16
Navigating to a folder using the VS Code terminal . . . 17

Starting a new oTree project: `otree startproject` . . . 18
What just got created? . . . 19
Starting a new app: `otree startapp` . . . 20
Running the server: `otree devserver` . . . 20
Resetting the database: `otree resetdb` . . . 21
Zipping your project: `otree zip` . . . 21
Summary of terminal commands . . . 22

Inside an oTree Project: Folder Structure and `settings.py` **23**
The oTree project folder . . . 23
The `settings.py` file . . . 25
`SESSION_CONFIGS`: Your session list . . . 26
`SESSION_CONFIG_DEFAULTS`: Project-wide defaults . . . 27
`PARTICIPANT_FIELDS` and `SESSION_FIELDS` . . . 27
`DEBUG` . . . 28
What you should never touch . . . 29
Summary . . . 31

Your First App: Adding, Registering, and Understanding `__init__.py` **32**
Step 1: Download and add the `demographic_survey` app . . . 32
Step 2: Register the app in `SESSION_CONFIGS` . . . 33
Step 3: Demo the app . . . 34
Step 4: Open `__init__.py` and understand the structure . . . 34
The import statement . . . 37
The `doc` string . . . 37
Class `C`: Constants . . . 38
Data model classes: `Subsession`, `Group`, and `Player` . . . 39
The `Player` class and model fields . . . 39
Page classes . . . 42
The `page_sequence` list . . . 43
Summary: The anatomy of `__init__.py` . . . 44

Session 2 **45**

Session 3 **46**

Session 4 **47**

Useful Resources **48**
oTree . . . 48
Python . . . 49

References **50**

Welcome

This book serves as an A to Z resource for beginners learning to use oTree to conduct online behavioral experiments and surveys involving human subjects. No prior programming experience is assumed. It is designed to accompany ECON 227B Experimental Economics II lectures and course materials. The detailed syllabus and other course materials are available on the Canvas page for this course.

In this book, you will be introduced to coding simple experiments in the form of "applications" using oTree and deploying them on the Heroku server using oTree Hub. We will begin by installing oTree and Visual Studio Code on your personal laptops and familiarizing ourselves with the VS Code environment. Topics covered include creating an oTree application, executing common oTree tasks, testing code on a local server, running a session with human subjects, coding best practices, and using AI effectively to build your code. This book will also provide a curated list of resources by oTree creators and other users that you can draw on for your individual projects. The point is not to reinvent the wheel, but to stand on the shoulders of giants!

Prerequisites

Familiarity with Python is helpful but not required — we will build everything from scratch together. If you have never written a line of code before, do not worry. By the end of this course you will be comfortable navigating a coding environment and building your own experiments.

A Note on This Book

This book will grow alongside the course. Links to reference code and additional materials will be added as we progress through each session. If you are browsing ahead and find session pages that appear empty or incomplete, that is expected — content will be added as we reach those sessions in class.

Let's Get Started!

This section walks you through setting up your development environment before using oTree. Before we meet on April 1st for our first lab session, we request you to download and install the relevant software requirements.

Please ensure you have the following installed on your personal device:

1. Python — A general-purpose programming language.
2. oTree — An open-source Python-based framework used to build online interactive experiments.
3. Visual Studio Code — The Integrated Development Environment (IDE) we will be using for oTree.

Step-by-step instructions for each are provided in the sections below.

Setting Up Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight code editor we will use throughout this course to write and run oTree code. Follow the instructions below for your operating system.

macOS

Installation

1. Go to https://code.visualstudio.com/.
2. Click **Download for Mac**.
3. Once downloaded, open the `.zip` file — this extracts **Visual Studio Code.app**.
4. Drag **Visual Studio Code.app** into your **Applications** folder.
5. Open VS Code from your Applications folder or Spotlight Search (`Cmd + Space`, type "VS Code").

> **Tip**
>
> To keep VS Code easily accessible, right-click its icon in the Dock and select **Options → Keep in Dock**.

Windows

Installation

1. Go to https://code.visualstudio.com/.
2. Click **Download for Windows**.
3. Run the downloaded `.exe` installer.
4. Follow the setup wizard and click **Install**, then **Finish**.
5. Launch VS Code from the Start Menu or Desktop shortcut.

Installing Extensions

Extensions add functionality to VS Code. For this course, you will need the following:

How to Install an Extension

1. Open VS Code.
2. Click the **Extensions** icon in the left sidebar (or press `Ctrl+Shift+X` on Windows / `Cmd+Shift+X` on Mac).
3. Type the extension name in the search bar.
4. Click **Install**.

Required Extensions

Extension	Publisher	Purpose
Python	Microsoft	Core Python language support
Pylance	Microsoft	Improved Python code suggestions and error detection
Python Debugger	Microsoft	Step-through debugging for your oTree code

Recommended Extensions

Extension	Publisher	Purpose
Django	Baptiste Darthenay	Syntax highlighting for oTree HTML templates, which are Django-based
Python Environments	Microsoft	Manage Python virtual environments for your oTree projects
vscode-icons	VSCode Icons Team	Adds file and folder icons for easier project navigation

> **Note**
>
> If you already have VS Code installed, open the Extensions panel and check which of these you already have. You only need to install the missing ones.

Installing Python and oTree

oTree is built on Python, so we need to install Python first before installing oTree. Follow the steps below for your operating system.

Step 1: Check if Python is Already Installed

Before downloading anything, check if Python is already on your system. Open your Terminal (macOS) or Command Prompt (Windows) and run:

macOS — run in Terminal:

```
python3 --version
```

Windows — run in Command Prompt:

```
python --version
```

If you see something like `Python 3.10.x` or higher, you already have Python and can **skip to Step 2**.

> **Note**
>
> If your terminal prompt starts with `(base)`, you have Anaconda or Miniconda installed. This is perfectly fine — Python is already included and you can skip directly to Step 2.

Step 2: Install Python (Only if Needed)

macOS

1. Go to https://www.python.org/downloads/.
2. Click **Download Python** — it will suggest the latest version.
3. Open the downloaded `.pkg` file and follow the installer.
4. Once installed, verify by running:

```
python3 --version
```

You should see something like `Python 3.12.x`. If you do, Python is successfully installed.

Windows

1. Go to https://www.python.org/downloads/.
2. Click **Download Python** — it will suggest the latest version.
3. Run the downloaded `.exe` installer.
4. **Important:** On the first screen, check the box that says **Add Python to PATH** before clicking Install Now.

> ⚠ Warning
>
> If you skip checking **Add Python to PATH**, Python will not be recognized in the terminal and the next steps will not work. If you already installed Python without this box checked, uninstall it and reinstall with the box checked.

5. Once installed, verify by running:

```
python --version
```

You should see something like `Python 3.12.x`. If you do, Python is successfully installed.

Step 3: Install oTree

Once Python is confirmed, install oTree using `pip`, Python's built-in package installer.

macOS — run in Terminal:

```
pip3 install otree --upgrade
```

Windows — run in Command Prompt:

```
pip install otree --upgrade
```

This may take a minute or two as pip downloads and installs oTree and its dependencies. The `--upgrade` flag ensures you get the latest version even if oTree was previously installed on your system.

Step 4: Verify oTree Installation

Once the installation is complete, verify it by running the following in your Terminal or Command Prompt:

```
oTree --version
```

You should see `6.0.13` or similar. If you do, you are all set!

Troubleshooting

Problem	Fix
`python3: command not found` (macOS)	Install Python from python.org
`python: command not found` (Windows)	Reinstall Python and check **Add Python to PATH**
`pip3: command not found` (macOS)	Run `python3 -m ensurepip` in Terminal
`oTree: command not found`	Run `pip3 install otree --upgrade` again and restart Terminal

> **Tip**
>
> If you are stuck before the first session, you can use ZotGPT — UCI's free AI platform — to troubleshoot. Select the latest Claude model available and describe your error message. It is particularly good at solving software installation issues.
> You can also bring your laptop to the first lab session on April 1st and we will troubleshoot together.

Session 1

In this session, we build the foundation for everything that follows in this course. We start by getting comfortable with VS Code and the terminal, then move into creating and understanding the structure of an oTree project, and finish with a hands-on exercise where you will download, configure, and run a real oTree app.

By the end of this session, you will be able to:

- Navigate your file system using the terminal and create a new oTree project and app from scratch.
- Read and make sense of what is inside an oTree project folder, and understand the key components of the `settings.py` file.
- Download an app shared by someone else, add it to your project, update `settings.py` to include it, and run a working demo in your browser.

This session has three parts:

Part	Title	What you will do
Getting Started	Getting Started: VS Code, Projects, and Apps	Learn the core terminal commands for working with oTree
Inside an oTree Project	Inside an oTree Project: Folder Structure and `settings.py`	Understand what oTree creates and how to configure it
Hands-On	Hands-On: Adding and Running Your First App	Download, configure, and demo a shared oTree app

Getting Started: VS Code, Projects, and Apps

This part covers the core terminal commands you will use every time you work in oTree. By the end of this part, you will be able to create a new oTree project and app from scratch, run the development server, and navigate your project folder in VS Code.

Opening VS Code and navigating to the right folder

Before creating an oTree project, decide where on your computer you want to store your work. A good habit is to keep all your oTree projects inside a dedicated folder — for example, a folder called `oTree_lab` inside your Documents folder.

To open that folder in VS Code:

1. Open VS Code
2. Click **File** → **Open Folder**
3. Navigate to your chosen folder and click **Open**

You should now see the folder listed in the Explorer panel on the left side of the screen.

Opening the terminal in VS Code

Almost everything in oTree is done through the terminal. In VS Code, open a terminal by clicking **Terminal** → **New Terminal** from the top menu bar. A terminal panel will appear at the bottom of the screen.

> **What is the terminal?**
>
> The terminal is a text-based interface where you type commands directly to your computer. You will use it constantly in oTree — to navigate between folders, create projects and apps, and run the server.

Navigating folders: `cd`

The terminal always has a "current directory" — the folder it is currently pointed at. Before running any oTree commands, make sure the terminal is pointed at the right place.

The command `cd` (short for "change directory") is how you move around.

Navigate into a folder:

```
cd Documents/oTree_lab
```

Go back to your home directory:

```
cd ~
```

> ⚠ Avoid spaces in folder names
>
> If any folder in your path has a space in its name, wrap the entire path in quotes:
>
> ```
> cd "Documents/my projects/oTree_lab"
> ```
>
> The safest habit is to never use spaces in folder or file names at all — use underscores instead (e.g., `oTree_lab`).

Finding your working directory: `pwd`

Before running any oTree command, you need to know exactly where the terminal is pointed. The command `pwd` (print working directory) tells you the full path of your current location.

```
pwd
```

You will see output like this:

```
/Users/nidhipandey/Documents/GitHub/all-otree-code
```

This is the single most useful command for diagnosing navigation errors. If a `cd` command fails with "no such file or directory," run `pwd` first to confirm where you actually are before trying again.

> ⚠ The Explorer panel and the terminal are not always in sync
>
> The VS Code Explorer panel shows the folder you have **open in VS Code**. The terminal shows the folder it is **currently pointed at**. These are two separate things and they can easily get out of sync — especially if you open the terminal from a different location or run `cd ~` and navigate elsewhere.
> Always check `pwd` if the terminal and the Explorer panel seem to be showing different things.

Navigating to a folder using the VS Code terminal

The safest and most reliable way to make sure the terminal is pointed at the right folder is to let VS Code do it for you.

The recommended workflow:

1. Open VS Code
2. Click **File** → **Open Folder** and select the folder you want to work in — for example, `all-otree-code`
3. Click **Terminal** → **New Terminal**

VS Code will automatically open the terminal pointed at the folder you have open in the Explorer panel. You will see the folder name in the terminal prompt confirming this.

From there, if you want to navigate into a subfolder — for example `oTree_lab` — just run:

```
cd oTree_lab
```

This works every time because the terminal is already inside the parent folder. You do not need to type out the full path.

> ❗ Always open a new terminal from inside VS Code
>
> If you open a terminal from outside VS Code — for example from macOS Spotlight or the Applications folder — it will start pointed at your home directory `~`, not at your project folder. You will then have to navigate manually using the full path, which is easy to get wrong.
> The habit to build: **open VS Code first, open the right folder, then open the terminal from inside VS Code.**

When `cd` says "no such file or directory"

This error does not always mean the folder does not exist. It usually means the terminal cannot see it from its current location. Run `pwd` to check where you are, then build the correct path from there. For example, if `pwd` returns `/Users/yourname/Documents/GitHub/all-otree-code`, then a folder called `oTree_lab` inside it is reachable with:

```
cd oTree_lab
```

But a folder somewhere else entirely on your computer will not be reachable with just its name. You must navigate to its parent folder first, or use the full path:

```
cd /Users/yourname/Documents/GitHub/all-otree-code/oTree_lab
```

Starting a new oTree project: `otree startproject`

Project vs. App — know the difference

An oTree **project** is the top-level container for your experiment. It holds the `settings.py` file and all of your apps. You will typically only create one project per course or research project. This is a rare task; most of the times you would be creating new apps within an existing project.
An oTree **app** is a single task or module within your experiment (e.g., a survey, a game, a payment screen). You will create new apps regularly.
Make sure you know which one you need before running a command.

Navigate to your desired folder in the terminal, then run:

```
otree startproject PROJECT_NAME
```

Replace `PROJECT_NAME` with a name of your choice — no spaces or special characters. For example:

```
otree startproject oTree_lab
```

oTree will ask whether you want sample games included:

```
Include sample games? (y or n):
```

Type y and press Enter. This is useful for your first project as it gives you working examples to explore.

Once the project is created, navigate into it:

```
cd oTree_lab
```

What just got created?

Click on the image below to see what the oTree project folder that you created looks like.

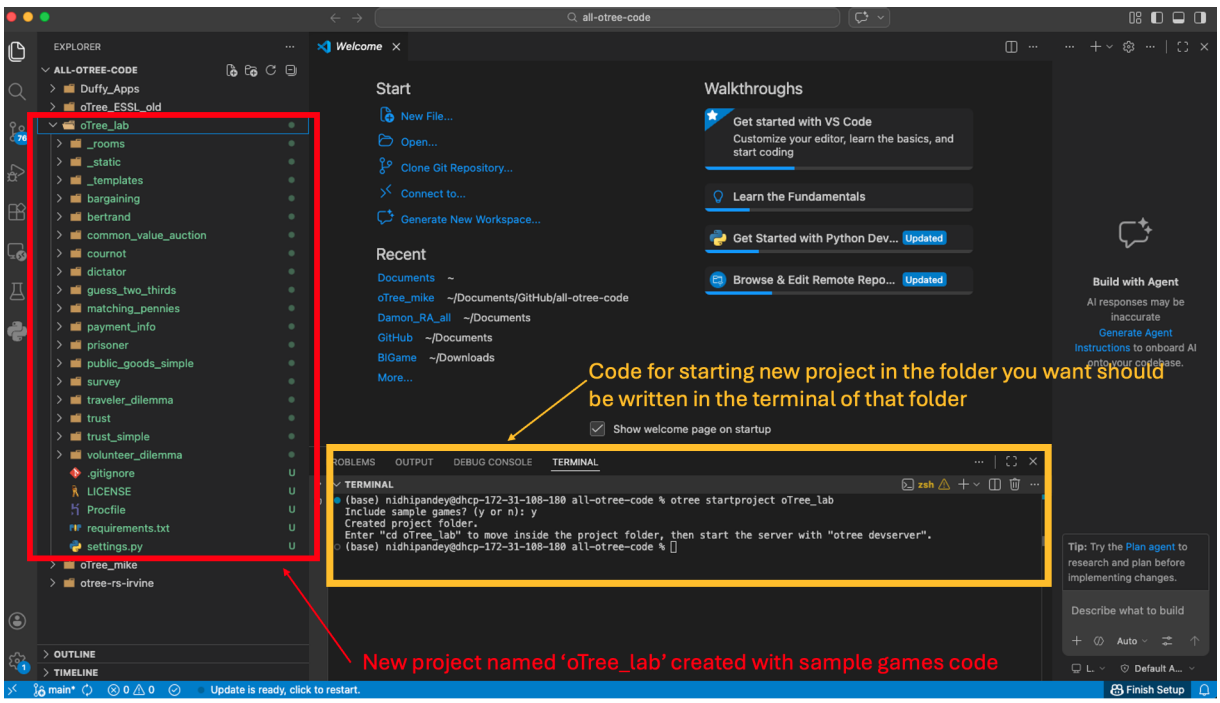

Figure 0.1: VS Code after running `otree startproject oTree_lab`. The Explorer panel (left) shows the newly created project folder containing the `settings.py` file and all sample app folders. The terminal at the bottom shows the full sequence of commands run to create and navigate into the project. Note: the code for starting a new project should be written in the terminal of the folder you want the project to live in.

Open the Explorer panel in VS Code and look at your new project folder. Here are the key things to know:

Item	What it is
`settings.py`	The control center for the entire project. You will edit this regularly.
`_rooms/`	Where room configuration files go. Do not touch unless instructed.
`_static/`	Where shared static files (images, CSS, JS) go.
`_templates/`	Where shared HTML templates go.
Sample app folders	The sample games oTree installed for you.

Starting a new app: `otree startapp`

Make sure your terminal is pointed at your project folder, then run:

```
otree startapp APP_NAME
```

For example:

```
otree startapp my_survey
```

This creates a new folder called `my_survey` inside your project. Inside it you will find:

- `__init__.py` — the Python file where all the logic for your app lives
- No HTML files yet — you will create those yourself as you build out your pages

> **Name your apps carefully**
>
> App names become part of the URL that participants see during your experiment. Use short, lowercase names with underscores and no special characters. Avoid names that reveal anything about your experimental design (e.g., do not call an app `treatment_group`).

Running the server: `otree devserver`

To preview your project in the browser, run:

```
otree devserver
```

You will see output like this in the terminal:

```
Open your browser to http://localhost:8000/
To quit the server, press Control+C.
```

Open your browser and go to `http://localhost:8000/demo` to see the oTree Demo page. All the sample games will be listed there. Click on any one to try it out.

> ⚠ Always quit the server when you are done
>
> When you are finished, go back to the terminal and press **Ctrl + C** to stop the server. Do not just close the browser window — always quit the server first.

Resetting the database: `otree resetdb`

Whenever you make changes to model variables in your app or update `settings.py`, you need to reset the database before running the server again:

```
otree resetdb
```

oTree will ask you to confirm:

```
This will delete and recreate your database.
Proceed? (y or n): y
```

> ❗ Do not reset if you have real session data
>
> Resetting the database permanently deletes all session data. If you have collected data from real participants, download it **before** running `otree resetdb`. You cannot recover it afterward.

Zipping your project: `otree zip`

The `otree zip` command packages your entire oTree project into a single `.otreezip` file. This is useful for sharing your project with collaborators, submitting it for a class, or uploading it to a server for deployment.

Make sure your terminal is pointed at your project folder, then run:

```
otree zip
```

oTree will create a file called `PROJECT_NAME.otreezip` in the same folder. For example, if your project is called `my_experiment`, the output will be:

```
Created my_experiment.otreezip
```

You can share this file directly with anyone who has oTree installed. To unzip and run it, they simply run:

```
otree unzip my_experiment.otreezip
```

> When to zip
>
> Zip your project whenever you want to share it or submit it. It is also a good habit to zip and save a copy of your project before making major changes — think of it as a quick manual backup.

Summary of terminal commands

Command	What it does
`pwd`	Print the full path of your current directory
`cd FOLDER_NAME`	Navigate into a folder
`cd ~`	Return to your home directory
`otree startproject NAME`	Create a new oTree project
`otree startapp NAME`	Create a new app inside a project
`otree devserver`	Start the local development server
`otree resetdb`	Reset the database
`otree zip`	Package the project into a `.otreezip` file
`otree unzip FILE.otreezip`	Unpack a zipped project
`Ctrl + C`	Stop the server

Inside an oTree Project: Folder Structure and `settings.py`

Now that you have created your first oTree project, it is time to understand what is actually inside it. Every decision about how your experiment runs — which apps are included, how many participants can demo it, what the payment structure looks like — flows through one file: `settings.py`. This part explains the project folder layout and walks through the key components of `settings.py` so you know what to edit, what to leave alone, and why.

The oTree project folder

When you run `otree startproject` and include sample games, oTree generates a ready-to-use project folder. Click on the image below to see what that looks like in VS Code.

Here is a breakdown of every item you will see in a new oTree project:

Item	What it is	Do you edit it?
`settings.py`	The control center for the entire project	Yes — regularly
`_rooms/`	Stores room configuration files for live sessions	Only when instructed
`_static/`	Shared static files (images, CSS, JS) used across apps	As needed
`_templates/`	Shared HTML templates used across apps	As needed
App folders (e.g., `survey/`)	Each folder is one oTree app	Yes — this is where you build
`requirements.txt`	Lists Python packages the project depends on	Rarely
`Procfile`	Server configuration file	Do not touch
`LICENSE`	License file	Do not touch

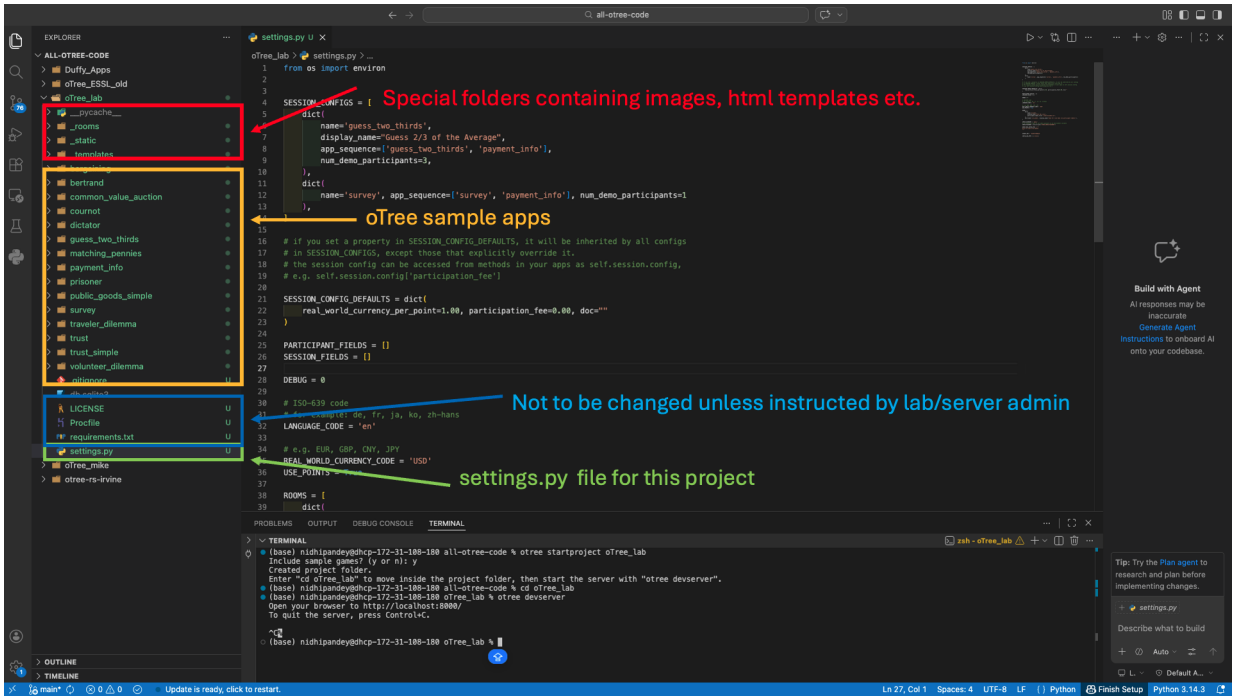

Figure 0.1: The oTree project folder `oTree_lab` open in VS Code. The Explorer panel shows the full contents of the project: the special system folders (`_rooms`, `_static`, `_templates`), the pre-installed sample app folders (such as `bargaining`, `prisoner`, `survey`, and others), and the `settings.py` file at the bottom of the list. The terminal confirms that the project was created and the working directory is now inside `oTree_lab`.

> ⚠ Special folders vs. app folders
>
> The three folders that begin with an underscore — `_rooms/`, `_static/`, and `_templates/` — are system folders, not apps. Do not confuse them with your app folders. Never delete or rename them unless instructed by your lab administrator.

The `settings.py` file

The `settings.py` file is the control center for your entire oTree project. Every app you want to run, every payment parameter, and every session configuration lives here. You will edit this file regularly throughout the course.

Click on the image below to see what a typical `settings.py` file looks like.

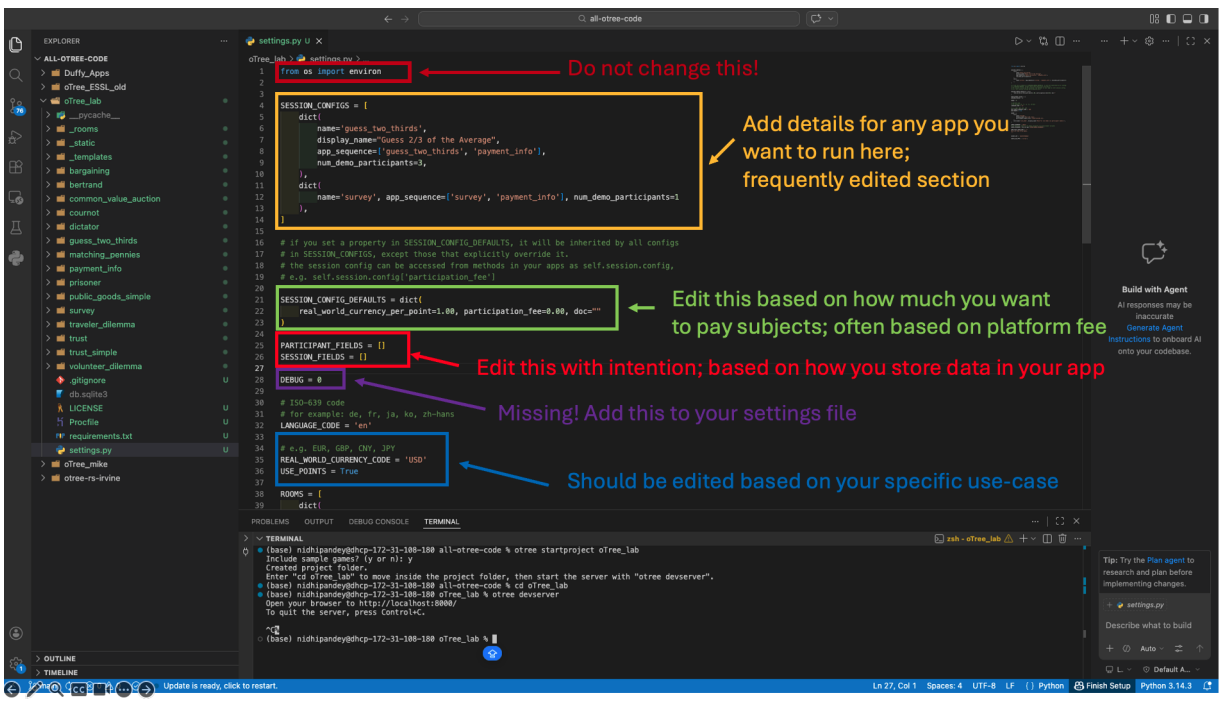

Figure 0.2: The `settings.py` file for an oTree project open in VS Code. The file contains several distinct sections: `SESSION_CONFIGS` (edited regularly), `SESSION_CONFIG_DEFAULTS` (edit with intention as needed), `PARTICIPANT_FIELDS` and `SESSION_FIELDS` (edited with intention as needed), and system-level settings such as `SECRET_KEY` and `INSTALLED_APPS` (never touch these). Each section serves a different purpose in controlling how your experiment runs.

`SESSION_CONFIGS`: Your session list

`SESSION_CONFIGS` is a Python list of dictionaries. Each dictionary defines one session — one experiment you can run or demo from the admin interface. Think of it as the menu of sessions available in your project.

Here is what a `SESSION_CONFIGS` list looks like:

```
SESSION_CONFIGS = [
    dict(
        name='guess_two_thirds',
        display_name="Guess 2/3 of the Average",
        app_sequence=['guess_two_thirds', 'payment_info'],
        num_demo_participants=3,
    ),
    dict(
        name='survey',
        app_sequence=['survey', 'payment_info'],
        num_demo_participants=1,
    ),
]
```

Every session dictionary requires three key-value pairs:

Key	What it does
`name`	A unique internal identifier for the session. No spaces or special characters.
`app_sequence`	A Python list of app names in the order participants will complete them.
`num_demo_participants`	How many participants are created when you run a demo of this session.

> **Choosing `num_demo_participants`**
>
> Set this number based on your experimental design. If an app groups participants in pairs, use a multiple of 2. If it groups them in threes, use a multiple of 3. For individual decision-making tasks, 1 is enough to test the app.

You can also include optional keys in any session dictionary:

Optional key	What it does
`display_name`	The name shown in the admin interface. If omitted, `name` is used.
`real_world_currency_per_point`	Overrides the default exchange rate for this session only.
`participation_fee`	Overrides the default participation fee for this session only.

`SESSION_CONFIG_DEFAULTS`: Project-wide defaults

`SESSION_CONFIG_DEFAULTS` is a single dictionary that sets default values inherited by all sessions unless a session dictionary explicitly overrides them. The two most commonly used defaults are `real_world_currency_per_point` and `participation_fee`.

```
SESSION_CONFIG_DEFAULTS = dict(
    real_world_currency_per_point=1.00,
    participation_fee=10.00,
    doc=""
)
```

If you want one specific session to use a different exchange rate or participation fee, simply add that key to that session's dictionary in `SESSION_CONFIGS` and it will override the default for that session only.

> **!** Lab participation fees
>
> If you run sessions through the ESSL lab, you are bound by the lab's participation fee policy. Make sure the `participation_fee` in your settings reflects the approved rate before running any real sessions.

`PARTICIPANT_FIELDS` and `SESSION_FIELDS`

These two settings are lists that become relevant in more advanced experiments where you need to carry data from one app to the next — for example, storing a participant's treatment assignment in the first app and reading it back in the payment screen.

- Use `PARTICIPANT_FIELDS` for data that is specific to an individual participant (e.g., their assigned treatment condition).
- Use `SESSION_FIELDS` for data that applies equally to everyone in the session.

For now, you can leave these as empty lists:

```
PARTICIPANT_FIELDS = []
SESSION_FIELDS = []
```

You will learn how to use them properly in later sessions when your experiments involve multiple apps.

`DEBUG`

You do not currently have this in your settings.py file. Add DEBUG = 0 to your file settings.py under `SESSION_FIELDS`:

```
PARTICIPANT_FIELDS = []
SESSION_FIELDS = []

DEBUG = 0
```

What `DEBUG` does

`DEBUG` controls whether oTree runs in development mode or production mode.

Value	Mode	What it means
`DEBUG = 1`	Development mode	Extra tools and error information are turned on
`DEBUG = 0`	Production mode	Clean participant-facing interface, no extra tools

Why `DEBUG = 1` is useful when building your app

When you are writing and testing your code, setting `DEBUG = 1` gives you two very useful advantages:

1. **Auto-advance buttons** — On every page, oTree adds a small button that lets you skip past that page without manually filling in any form fields — pages are submitted with empty or default values instead. This lets you click through the entire experiment in seconds to check that the page flow works correctly, without having to fill in every input every time.
2. **Detailed error messages** — If your Python code crashes, oTree shows you a full error traceback directly in the browser instead of just a blank page. This makes it much faster to find and fix problems in your code.

These two features make the development cycle dramatically faster. Instead of filling in every form field on every page just to check one thing at the end of the experiment, you can click through the whole app in a few seconds.

Why you must switch to `DEBUG = 0` before running real sessions

When `DEBUG = 1`, participants can also see the auto-advance buttons and skip past pages without submitting their answers. This means your data will be incomplete or entirely missing. Always confirm that `DEBUG = 0` before inviting any real participants.

> ⚠ Do not run real sessions with `DEBUG = 1`
>
> If `DEBUG = 1` is on during a live session, participants will see a skip button on every page that lets them bypass your forms — if they use it, their answers will not be recorded. Always confirm `DEBUG = 0` before starting any real data collection.

> 💡 A useful development habit
>
> While building and testing your app: set `DEBUG = 1`. Before running any real session: set `DEBUG = 0`. This single switch is one of the most common sources of data collection errors for new oTree users — do not forget it.

What you should never touch

Some parts of `settings.py` must not be modified under any circumstances:

Setting	Why you leave it alone
`from os import environ`	System import required by oTree — removing it breaks the project
`SECRET_KEY`	Security key for the project — changing it will break everything
`INSTALLED_APPS`	Registers oTree with Django — do not modify
`ADMIN_PASSWORD`	Managed via environment variables by your administrator

> ⚠ When in doubt, do not touch it
>
> If a line in `settings.py` is not covered in this guide, leave it exactly as it is. Only edit the parts you understand. When in doubt, ask before making any changes.

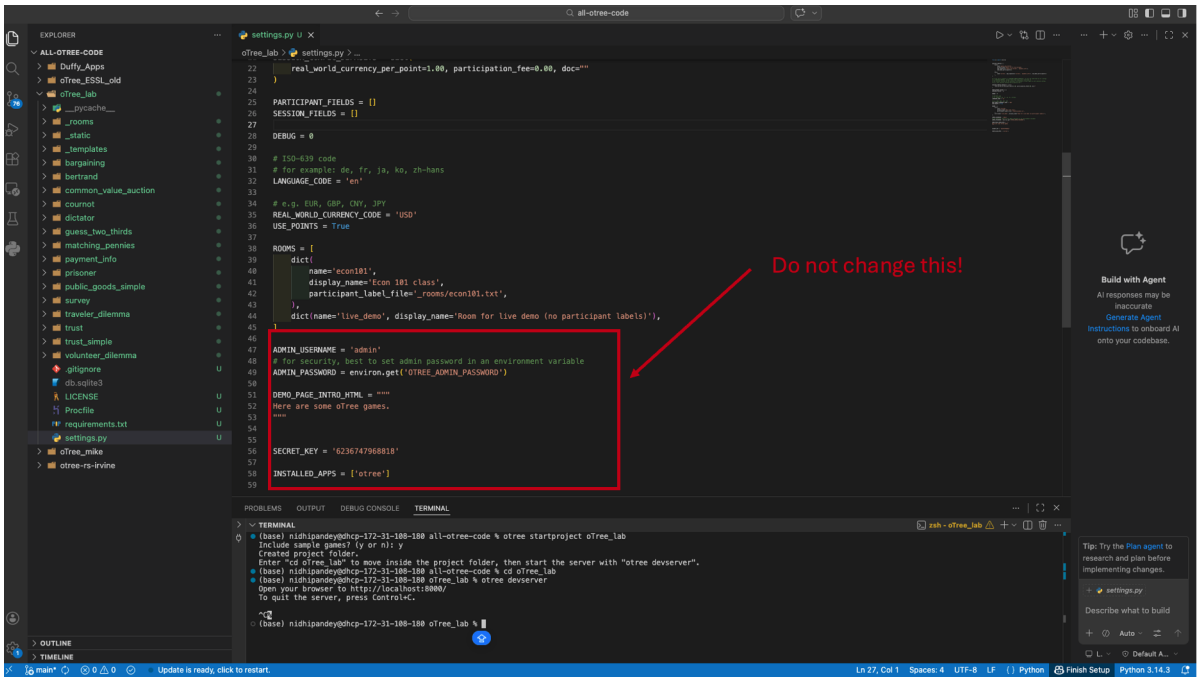

Figure 0.3: The end of the `settings.py` file for an oTree project open in VS Code. Do not change system-level settings such as `SECRET_KEY` and `INSTALLED_APPS`. Each section serves a different purpose in controlling how your experiment runs.

Summary

Section	What it controls	How often you edit it
`SESSION_CONFIGS`	Which sessions and apps are available	Regularly
`SESSION_CONFIG_DEFAULTS`	Default payment and session parameters	Occasionally
`PARTICIPANT_FIELDS`	Cross-app participant-level data	When needed
`SESSION_FIELDS`	Cross-app session-level data	When needed
`USE_POINTS`	If `True`, earnings are shown in points and converted using the exchange rate. If `False`, real currency is used directly	When needed — based on your design
`DEBUG`	`0` = production mode (normal). `1` = debug mode (shows extra info on participant pages)	Leave at `0` unless developing app or troubleshooting
System settings	Security, server configuration	Never

Your First App: Adding, Registering, and Understanding `__init__.py`

In this session you will add a pre-built app to your oTree project, register it in `settings.py`, demo it in the browser, and then open the code to understand how an oTree app is structured from the inside. By the end of this part you will be able to read and navigate any `__init__.py` file and understand what each component does.

Step 1: Download and add the `demographic_survey` app

The example code for this course is stored in a shared Google Drive folder. Navigate to the link below and download the file called `demographic_survey.zip`:

Example Code — Google Drive

Once downloaded:

1. Locate `demographic_survey.zip` in your Downloads folder
2. Double-click the file to unzip it — this will produce a folder called `demographic_survey`
3. Move the `demographic_survey` folder into your oTree project folder — for example, into `oTree_lab`

After moving it, open VS Code and check the Explorer panel. You should now see `demographic_survey` listed alongside the other app folders inside your project.

> ⚠ The folder must be inside the project folder
>
> The `demographic_survey` folder must sit directly inside your oTree project folder — at the same level as `settings.py` and the other app folders. If it is placed anywhere else, oTree will not be able to find it.

Step 2: Register the app in `SESSION_CONFIGS`

Adding the folder to your project is not enough on its own. oTree only runs apps that are listed in `SESSION_CONFIGS` inside `settings.py`. Open `settings.py` and add the following dictionary to your `SESSION_CONFIGS` list:

```
SESSION_CONFIGS = [
    # ... your existing session dictionaries ...

    dict(
        name='survey_leveled_up',
        display_name="End of experiment questionnaires",
        app_sequence=['demographic_survey'],
        num_demo_participants=1,
        doc="Standard end of experiment questionnaires."
    ),
]
```

Notice the five key-value pairs in this dictionary:

Key	Value	What it does
`name`	`'survey_leveled_up'`	Internal identifier used by the server
`display_name`	`"End of experiment questionnaires"`	The name shown in the admin interface
`app_sequence`	`['demographic_survey']`	The list of apps in this session — just one here
`num_demo_participants`	`1`	One participant is enough to demo a survey
`doc`	`"Standard end of experiment questionnaires."`	A short note describing the session — optional but good practice

Save `settings.py` after making the change.

> **❗ Reset the database after editing `settings.py`**
>
> Any time you edit `settings.py` — including adding a new session — you must reset the database before running the server. In the terminal, make sure you are inside your project folder, then run:
>
> ```
> otree resetdb
> ```
>
> Confirm with `y` when prompted.

Step 3: Demo the app

With the database reset, start the development server:

```
otree devserver
```

Open your browser and go to `http://localhost:8000/demo`. You should now see `End of experiment questionnaires` listed on the Demo page. Click on it to launch a demo session and work through the app as a participant would. When you are done, press **Ctrl + C** in the terminal to stop the server.

Step 4: Open `__init__.py` and understand the structure

Every oTree app has exactly one `__init__.py` file. This single Python file contains everything the app needs to run: the constants, the data models, the page logic, and the page sequence. Open `demographic_survey/__init__.py` in VS Code. Here is the full file:

```
from otree.api import *

doc = """
Standard, end-of-session survey.
Includes Cognitive Reflection Task and demographic survey.
"""

class C(BaseConstants):
    NAME_IN_URL = 'survey_experiment'
    PLAYERS_PER_GROUP = None
    NUM_ROUNDS = 1


class Subsession(BaseSubsession):
    pass


class Group(BaseGroup):
    pass


class Player(BasePlayer):
    age = models.IntegerField(
        label='What is your age?',
```

```
        min=18,
        max=99,
    )
    gender = models.StringField(
        choices=[['Male', 'Male'], ['Female', 'Female'],
                 ['Non-Binary', 'Non-Binary'],
                 ['NoState', 'Prefer not to state']],
        label='What is your gender?',
        widget=widgets.RadioSelect,
    )
    race = models.StringField(
        choices=[
            ['Black', 'African descent, Black, African-American'],
            ['Asian', 'Asian descent, Asian-American'],
            ['Indigenous', 'Indigenous, First Nations, Native American'],
            ['MiddleEastern', 'Middle Eastern or North African'],
            ['Hawaiian', 'Native Hawaiian or Pacific Islander'],
            ['White', 'White'],
            ['Other', 'Other'],
            ['Declined', 'Decline to state']
        ],
        label='What is your race?',
        widget=widgets.RadioSelect,
    )
    ethnicity = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Are you of Latinx/o/a or Hispanic origin?',
        widget=widgets.RadioSelect,
    )
    school = models.StringField(
        choices=[
            ['Social', 'School of Social Sciences'],
            ['Ecology', 'School of Social Ecology'],
            ['Law', 'School of Law'],
            ['Humanities', 'School of Humanities'],
            ['Education', 'School of Education'],
            ['Arts', 'Claire Trevor School of the Arts'],
            ['Business', 'The Paul Merage School of Business'],
            ['Biological', 'School of Biological Sciences'],
            ['Engineering', 'The Henry Samueli School of Engineering'],
            ['Computers', 'Donald Bren School of Information and Computer Sciences'],
            ['Interdisciplinary', 'Interdisciplinary Studies'],
```

```
            ['Physical', 'School of Physical Sciences'],
            ['Medicine', 'School of Medicine'],
            ['Nursing', 'Sue and Bill Gross School of Nursing'],
            ['Pharmaceutical', 'School of Pharmacy and Pharmaceutical Sciences'],
            ['Health', 'Program in Public Health']
        ],
        label='Choose the most appropriate option for your school below.',
        widget=widgets.RadioSelect,
    )
    field_of_study = models.StringField(
        label='What is your major or field of study?',
    )
    major_STEM = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Are you majoring in a STEM field?',
        widget=widgets.RadioSelect,
    )
    math_courses = models.IntegerField(
        label='How many Mathematics/Statistics courses have you taken since High School?',
        min=0,
        max=99,
    )
    english_is_first_language = models.BooleanField(
        choices=[[True, 'Yes'], [False, 'No']],
        label='Is English your first language?',
        widget=widgets.RadioSelect,
    )
    first_language = models.StringField(
        label='Please state your first language.',
        blank=True,
    )

    # ~~~ Cognitive Reflection Task

    crt_q1 = models.StringField(
        label="If John can drink one barrel of water in 6 days, and Mary can drink one barrel
    )
    crt_q2 = models.StringField(
        label="A man buys a pig for \$60, sells it for \$70, buys it back for \$80, and sells
    )
    crt_q3 = models.StringField(
        label="Jerry received both the 15th highest and the 15th lowest mark in the class. He
```

```
    )


# ~~~~~~~~~~~~~~~~~ PAGES

class Questionnaire_CRT(Page):
    form_model = 'player'
    form_fields = ['crt_q1', 'crt_q2', 'crt_q3']

    @staticmethod
    def is_displayed(player):
        return player.round_number == C.NUM_ROUNDS


class Demographics(Page):
    form_model = 'player'
    form_fields = ['age', 'gender', 'race', 'ethnicity', 'school',
                   'field_of_study', 'major_STEM', 'math_courses',
                   'english_is_first_language', 'first_language']


page_sequence = [Questionnaire_CRT, Demographics]
```

Now let us go through every part of this file.

The import statement

```
from otree.api import *
```

This line is the same in every single oTree app. It imports all of oTree's built-in classes, field types, widgets, and functions. You will never change this line.

The `doc` string

```
doc = """
Standard, end-of-session survey.
Includes Cognitive Reflection Task and demographic survey.
"""
```

The `doc` variable is an optional description of the app. It appears in the admin interface when you click on the app sequence for a session. It has no effect on how the app runs — it is purely informational. Write something useful here so that anyone looking at the admin interface understands what this app does without having to open the code.

Class `C`: Constants

```
class C(BaseConstants):
    NAME_IN_URL = 'survey_experiment'
    PLAYERS_PER_GROUP = None
    NUM_ROUNDS = 1
```

The `C` class holds all the fixed parameters of your app — values that are set before the session starts and do not change while it runs.

Constant	Value	What it means
`NAME_IN_URL`	`'survey_experiment'`	The string that appears in the participant's browser URL during the experiment
`PLAYERS_PER_GROUP`	`None`	Participants are not grouped — this is an individual task
`NUM_ROUNDS`	`1`	Participants go through the page sequence once

> ⚠ Participants can see `NAME_IN_URL`
>
> Whatever string you assign to `NAME_IN_URL` will be visible in the browser address bar during the experiment. Notice that this app uses `'survey_experiment'` rather than `'demographic_survey'` — this is deliberate. The name does not reveal what kind of survey is being administered. Do not use names that hint at the purpose or design of

your study.

Python naming conventions

Constant variables like `NAME_IN_URL` are written in `UPPERCASE_WITH_UNDERSCORES`. Non-constant variables and function names use `lowercase_with_underscores`. Class names use `PascalCase` (e.g., `BasePlayer`, `WaitPage`). VS Code's colour-coding works better when you follow these conventions.

Data model classes: `Subsession`, `Group`, and `Player`

Every `__init__.py` must define three data model classes. These represent the three levels of oTree's data structure and correspond directly to tables in the database.

```
class Subsession(BaseSubsession):
    pass

class Group(BaseGroup):
    pass
```

Both `Subsession` and `Group` contain only `pass` here because this app collects individual participant data only. No group-level or round-level variables are needed.

What `pass` means

In Python, `pass` is a placeholder that means "this class exists but has nothing extra defined in it." oTree requires all three model classes to be present in every `__init__.py`. If you are not adding any fields or methods to `Subsession` or `Group`, you must still include the class definition with `pass` underneath it. Removing either class entirely will crash the app.

The `Player` class and model fields

The `Player` class is where all the participant-level data lives. Every field you define here becomes a column in your exported data.

Field types used in this app

Field type	Used for	Example in this app
`models.IntegerField()`	Whole numbers	`age`, `math_courses`
`models.StringField()`	Text or multiple choice	`gender`, `race`, `school`, `field_of_study`
`models.BooleanField()`	True/False answers	`ethnicity`, `major_STEM`, `english_is_first_language`

Field arguments used in this app

Argument	What it does
`label`	The question or prompt shown to the participant on the page
`min` / `max`	Sets the minimum and maximum values for numeric fields
`choices`	A list of options for multiple choice questions
`widget`	Controls how the input is displayed — `widgets.RadioSelect` shows radio buttons
`blank`	If `True`, the field is optional and participants can skip it

How `choices` works

This is one of the most important things to understand about oTree model fields. The `choices` argument takes a list of two-item lists, where each inner list follows this pattern:

```
choices = [[recorded_in_database, shown_to_participant], ...]
```

Look at the `gender` field:

```
gender = models.StringField(
    choices=[['Male', 'Male'], ['Female', 'Female'],
             ['Non-Binary', 'Non-Binary'],
             ['NoState', 'Prefer not to state']],
    label='What is your gender?',
    widget=widgets.RadioSelect,
)
```

The participant sees four options on screen: Male, Female, Non-Binary, and Prefer not to state. But notice that `'NoState'` is what gets recorded in the database — not `'Prefer not to state'`. The first value in each pair is what oTree saves; the second is what the participant reads.

Now look at the `ethnicity` field:

```
ethnicity = models.BooleanField(
    choices=[[True, 'Yes'], [False, 'No']],
    label='Are you of Latinx/o/a or Hispanic origin?',
    widget=widgets.RadioSelect,
)
```

The participant sees Yes or No. The database records `True` or `False`. This is a clean and efficient way to store binary responses — it makes analysis much easier than storing the strings "Yes" and "No" directly.

> ❗ The `choices` format is always `[recorded, displayed]`
>
> The first value in each pair is stored in the database. The second value is what the participant sees. Never mix these up. If you do, your data will contain the wrong values and you will not catch the error until analysis.

The `blank=True` argument

```
first_language = models.StringField(
    label='Please state your first language.',
    blank=True,
)
```

By default, all form fields in oTree are required — participants cannot proceed to the next page without filling them in. Setting `blank=True` makes a field optional. Here `first_language` is optional because only participants who answered No to `english_is_first_language` need to complete it.

Section comments

```
# ~~~ Cognitive Reflection Task
```

This is just a Python comment — a line beginning with `#` that oTree ignores entirely. It is there purely for readability, separating the demographic fields from the CRT fields. Use comments freely in your own code to mark where one section ends and another begins.

Page classes

Each page in your app is defined as a class that inherits from `Page`. This app has two pages.

`Questionnaire_CRT`

```
class Questionnaire_CRT(Page):
    form_model = 'player'
    form_fields = ['crt_q1', 'crt_q2', 'crt_q3']

    @staticmethod
    def is_displayed(player):
        return player.round_number == C.NUM_ROUNDS
```

Attribute or method	What it does
`form_model = 'player'`	Tells oTree the form fields belong to the `Player` model
`form_fields`	Lists the three CRT fields that will appear as inputs on this page
`@staticmethod`	Required decorator before any Page-level method when working in VS Code or any IDE
`is_displayed`	A built-in oTree method that controls whether this page is shown to a participant. It returns `True` or `False`.

The `is_displayed` method here returns:

```
return player.round_number == C.NUM_ROUNDS
```

Since `C.NUM_ROUNDS = 1`, this evaluates to `player.round_number == 1`, which is always `True`

for a single-round app. In a multi-round app, this would restrict the page to only the final round. For now it is effectively always displayed.

> **`@staticmethod` is required for Page methods in VS Code**
>
> When you define any built-in oTree method inside a Page class — such as `is_displayed`, `vars_for_template`, or `before_next_page` — you must precede it with `@staticmethod`. This is because these methods are defined at the Page level rather than outside it. If you forget `@staticmethod`, the method will either not work or will produce an error.

`Demographics`

```
class Demographics(Page):
    form_model = 'player'
    form_fields = ['age', 'gender', 'race', 'ethnicity', 'school',
                   'field_of_study', 'major_STEM', 'math_courses',
                   'english_is_first_language', 'first_language']
```

This page has no special methods — just `form_model` and `form_fields`. All ten demographic fields will appear on this page in the order they are listed in `form_fields`. There is no `is_displayed` method, so this page is always shown.

The `page_sequence` list

```
page_sequence = [Questionnaire_CRT, Demographics]
```

This list controls the order in which participants see the pages. Participants will complete the CRT first, then the demographics questionnaire. If you swap the order here, they will see the demographics first.

> **Every page must be in `page_sequence`**
>
> If you define a Page class but forget to add it to `page_sequence`, oTree will not show it to participants. Conversely, if you list a page in `page_sequence` that does not have a corresponding class definition, the app will crash when you try to run it.

Summary: The anatomy of `__init__.py`

Section	What it contains	Required?
`from otree.api import *`	The oTree import statement	Always
`doc`	A short description of the app	Optional but recommended
`class C(BaseConstants)`	`NAME_IN_URL`, `PLAYERS_PER_GROUP`, `NUM_ROUNDS`	Always
`class Subsession`	Round-level logic and fields	Always — use `pass` if empty
`class Group`	Group-level logic and fields	Always — use `pass` if empty
`class Player`	Individual participant fields	Always — add your data fields here
Page classes	One class per page, with `form_model`, `form_fields`, and any methods	Always — at least one
`page_sequence`	The ordered list of page classes	Always

Session 2

Session 3

Session 4

Useful Resources

This is a curated list of resources to support your learning throughout this course. You do not need to master all of these — bookmark what is useful and return to it as needed.

oTree

The official oTree documentation is the most reliable resource for all questions about oTree. It is well-written, actively maintained, and covers fundamental concepts of what you will need.

Always check oTree's demo apps and common task recipes *before* you start writing code from scratch — sample code has likely already been written for what you are trying to do. The **Featured Projects** section on oTree Hub is a great place to start:

- **Game Theory Sample Games** — Classic games from economics and game theory (Prisoner's Dilemma, etc.). Useful for understanding how standard experiments are structured in oTree. Includes a live demo and source code.
- **More Demos** — A wide range of demo apps from different disciplines. A good starting point for finding code that is close to what you want to build. Includes a live demo and source code.
- **Snippets** — A collection of small recipes for common tasks in oTree. If you need to do something specific, check here first. Includes a live demo and source code(https://github.com/oTree-org/otree-snippets).

Both the old and the new oTree forums are useful sources of solutions to common problems. Note that the old forum contains solutions written in oTree's older `self` format, so some details may need to be adapted to the newer `no-self` format used in this course. See here for more details on how the new format differs from the old one.

Finally, the **Public projects** section on oTree Hub hosts code from other oTree users that is shared publicly.

Here is the complete list of oTree resources:

- Official oTree documentation
- Featured Projects on oTree Hub
- oTree's "Where do I put this code?" breakdown

- New oTree forum
- Old oTree forum
- Public projects

> Searching for oTree help
>
> When you run into a problem and are not sure where to start, try Googling your task or error message together with the word **"otree"** — for example, *"otree timer page"* or *"otree wait for all players"*. This will surface relevant results from the documentation, forums, and other oTree users.
> If you find a result on **Stack Overflow**, read the full discussion before implementing any solution — do not just copy the first answer you see. The comments often reveal important nuance, and your use-case may require a slightly different approach.

Python

Python is the backbone of your oTree app — it handles the logic, stores data, and communicates between the server and the participant's browser. A quick tip: use `print()` to see variable values and outputs in the terminal while you are debugging your code.

Web-based editors (for testing small bits of code)

The following editors are useful when you want to quickly test a small piece of Python code without running it through oTree. This list is adapted from resources originally compiled by Faith Bague-Sampson.

- Python Fiddle
- myCompiler: Python
- PYnative Online Python Editor

References